COMP 424 Final Project Game: *Ataxx*

Course Instructors: Jackie CK Cheung and Prakhar Ganesh
Due Date: Monday, Dec 1st, 2025, 8:59PM EST
Report Due Date: Wednesday, Dec 3, 2025 8:59PM EST
Source for this file: `https://www.overleaf.com/project/68e2b2f07781caef1e11d2bc`

This document contains the project's game description, your AI objectives, and the report instructions. This file will also serve as a template for you to start writing your report (if you want to use another tool like Word, that's OK, but match the format). The starter code, and programming instructions for your agent can be found at: `https://github.com/davaus80/COMP424-Fall2025`. This document is adapted from Prof. David Meger's 2024 specifications (see Section 9). **This project is to be completed in groups of three students**.

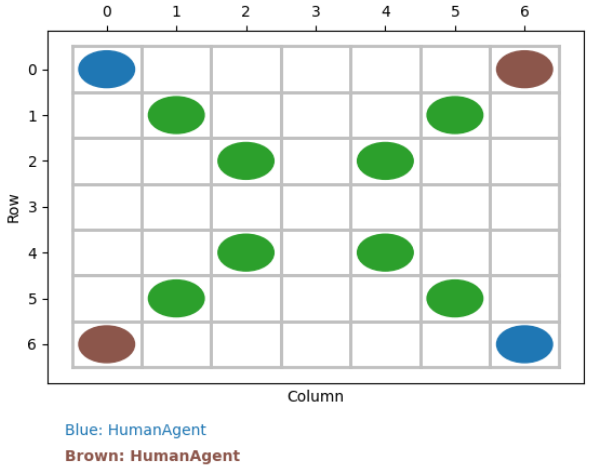


Figure 1: Gameboard. Players are in Blue and Brown, obstacles are in Green.

1. Ataxx: 2-Player Turn-Based Strategy Game

Ataxx is a 2-player turn-based strategy game played on a 7×7 board. Players take turns placing discs on the board with the objective of capturing the opponent's discs. A player wins when they have the majority of the discs on the board by the end of the game. See Figure 5 on the final page for example gameplay.

1.1 Setup

At the start of the game, a board layout is selected from a pool of possible layouts. Board layouts differ in the location of *obstacles*, on which discs cannot be placed. In all layouts, each player has two discs placed in opposite corners of the board. Player A uses brown

discs, and Player B uses blue discs. Players take turns placing discs, starting with Player A. Players can place discs in two ways: duplication or movement.

1.1.1 Duplication

A disc can *duplicate* into any adjacent tile that is not already occupied by a disc or an obstacle (see Figure 2). Diagonal tiles are considered adjacent. A disc remains on the source tile while a new disc is placed on the destination tile.

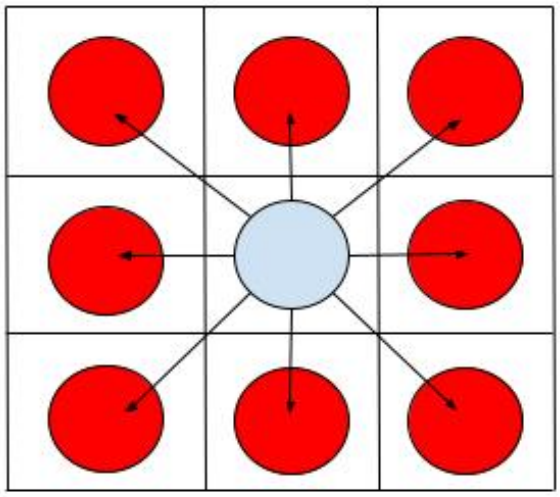


Figure 2: The central disc can duplicate to any of the 8 surrounding squares

1.1.2 Movement

A disc can *move* into any unoccupied square that is exactly two tiles (see Figure 3). This removes the disc from the source tile and places it on the destination tile. Since diagonal tiles are considered adjacent, that means that a disc may *move* to any tile that is two squares away, irrespective of direction.

1.1.3 Capture

When a disc is placed, all of the opponent's discs that are adjacent to it are captured and flipped to the current player's colour (see Figure 4). Note that this does not trigger a chain reaction (i.e. when an opponent's disc is flipped, we do not then flip the discs adjacent to it).

1.2 Objective

The game ends when neither player can make a valid move, which happens when all squares on the board are filled. The player with the majority of the discs on the board at the end of the game is declared the winner, receiving 1 point, with the loser receiving 0 points. In the case of a tie, both players are awarded 0.5 points.

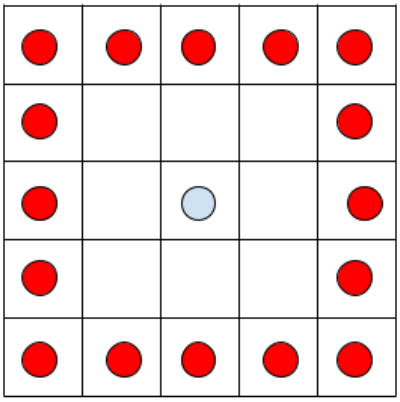


Figure 3: The central disc can move to any of the 16 squares which are two tiles away

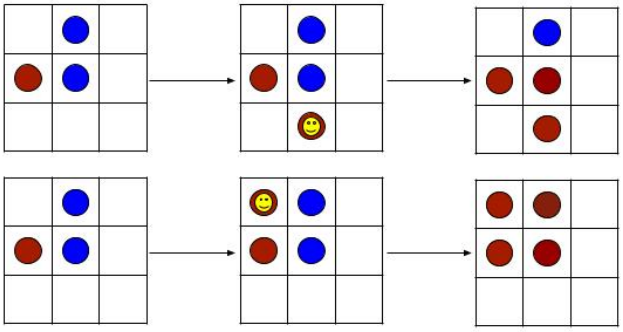


Figure 4: Two examples of a capture, where the smiley face indicates the duplicated disc.

1.3 Playing the game

At each turn, a player must provide two coordinates representing the row and column, (r, c) format, to indicate the positions of the source square and the destination square. For example, the input $(0, 1), (2, 3)$ indicates a move from $(0, 1)$ to $(2, 3)$. This would be typed into the command line interface as $0, 1, 2, 3$. You do not need to specify whether the move is a duplication or a movement - the game engine will handle this based on the distance between the source and destination points. The game engine will automatically flip the opponent's discs that are flanked by the placed disc in any direction (horizontal, vertical, or diagonal). If a player cannot make a valid move, their turn is automatically passed by the game logic, and the opponent will continue.

2. Assignment Details

In this final project, your task is to develop an *agent* to play the *Ataxx* game. We have developed a minimalistic game engine in Python, which you will extend to add your own *agents*.

2.1 Pre-requisites

For this project, you will need to implement an agent that plays Ataxx using Python. Specifically, we strongly recommend to brush up on Python 3 fundamentals before starting on your code.

2.2 Implementing your own agent

You need to write your own *agent* and submit it for the class project. Detailed instructions of various parts of the game are available in the **README.md** file, visible on the main Github website. Follow the steps there to implement and test your own agent. Be very precise about the instructions regarding your submitted student agent, because we need to be able to run your code automatically.
Important: You should not modify any other files apart from `student_agent.py` and your `authors.yaml` files, as we will copy your file into a "fresh" code checkout that includes all other parts of the game. Therefore, functionality changes you make in the simulator or world itself will be lost, and if this causes your code to crash, you will not receive performance grades. The provided helpers module has most of what we expect you to need related to interacting with the game, and you can also look at the sample agents to get started. Any further variables or functions you need should also be created within the student agent file. In the event that any update to the provided game code happens while the project is live, the TAs will announce the necessary steps in Ed to pull the starter code.

2.3 Analyzing and Improving Your Agent

Please read ahead and understand the report sub-sections in advance. If you simply win the tournament but have no idea what your agent is doing, you can get a full performance grade but still do poorly on the project. You should save at least a quarter of your overall effort simply to run the agents you've collected up to that point with printing, reporting or other diagnostics enabled so that you have a good idea why things work as they do and you have good things to say in the report component. Understanding and being able to clearly describe and compare the key points of these AI methods is also the key learning outcome of the project.

3. Report

You are required to write a report with a detailed explanation of your approach and reasoning. The report must be a typed PDF file, and should be free of spelling and grammar errors. The suggested length is between 4 and 8 pages, but the most important constraint is that the report needs to be clear and concise. You can use the source of this document[1] as

1. https://www.overleaf.com/project/68e2b2f07781caef1e11d2bc

a starting point for your report. Either create an overleaf account or download the Tex file to start from the same template and replace the content as appropriate. Any other method is also OK, but make sure that the formatting looks the same. The report must include the following required components:

- What is the strongest algorithm you found? Provide a brief overall summary and reasoning for the best approach. Skip details but give the reader an overview of what parts of the method are most important to achieve strong performance, what play quality you think you achieved and how you came to these conclusions (math/algorithm analysis, iterative design, reading sources, etc).
- A detailed explanation of your agent design, including a re-statement of any relevant general theoretical elements and algorithms (with citations), as well as specific details about how each of these maps to our game. Do not copy your code into this section, rather use plain English to describe code elements and data structures where they're relevant (2 pages).
- Analyze your agent's quantitative performance using criteria we mentioned in class. For each, state a numerical quantity or formula and give a 3-4 line text explanation (total 1 page):
    1. What **depth** (a.k.a. look-ahead) level does your agent achieve? Is it the same for all branches, or deeper in some cases than others? List elements of your approach that dealt with search depth.
    2. What **breadth** does your agent achieve? That is, how many moves do you consider at each level of play? Is this the same or different for the max and min player? Does your approach address move ordering, pruning or depth-first elements that may reduce the breadth?
    3. What impact does board layout have on your method (in particular, its look-ahead and search breadth, as listed above). You do not need to provide a board-by-board analysis - just the trends you noticed across all boards. Did you customize or analyze any method elements based on the board's layout?
    4. List the heuristics, pruning methods and move ordering approaches you tried, if any. Comment on the impact of each and why some were stronger than others.
    5. Predict your win-rates in the evaluation, against: (i) The random agent, (ii) an average human player, and (iii) your classmates' agents.
- A summary of the advantages and disadvantages of your approach, expected failure modes, or weaknesses of your program ($\frac{1}{2}$ page).
- A brief description of how you would go about further improving your player (e.g. by introducing other AI techniques, changing internal representations, etc.) These can be ideas that you ran out of time to implement (1 page).
- (Conditional) If you have used a significant input source including an LLM, Stack Overflow, past years' 424 code or a tutorial on game playing agents, you must describe that input in up to 1 extra page. For LLMs like ChatGPT, provide the first parts

of the prompt you used and outline how it was created. In all cases, describe what elements of your solution are exact duplicates of the input source and describe the changes you've made on top of these.

4. Submission

You may work in **teams of 3** to finish the project. For submission, follow the instructions down below.

1. Fill out your details in the `authors.yaml` file in the repository.
2. **The first student listed in the authors.yaml should submit** to the My Courses folder.
3. Please check the formatting and info in that file very carefully and do not submit the same project twice. If you do, we cannot ensure each partner will receive the same grade.

Important note: **Do not create a zip**, submit multiple files:

- Report as a PDF file (not the latex source)
- student_agent.py
- authors.yaml

You can make multiple submissions up to the deadline, but in this case we will be very strict about not accepting late code because we'll get the automated grading process running immediately after the deadline.

5. Grading Scheme

50% of the project grade will be allotted for performance in the tournament, and the other 50% will be based on your report.

5.1 Tournament Grading Scheme

The top scoring agent(s) will receive full marks for performance, other top agents in the tournament will receive marks according to a linear interpolation scheme based on the number of wins/losses they achieve. To get a passing grade on the performance portion, your agent must beat the random player.

The performance grade will be based on your agent's strength rating compared to other students and baselines implemented by the TAs. We will use several phases to group your agents by strength. A first pass is whether you can beat the random agent nearly 100% of the time. Next, a greedy one-step-lookahead agent called *greedy_corners_agent* that does beat the random agent, but it should be beaten by most of the search algorithms we've discussed in class. Finally, a tournament against the strongest set of agents within the class will be run to determine the rankings. You do not have to win the tournament to get a satisfactory grade, but the winning team(s) will receive 100% of the performance grade.

In all phases, each match will consist of N games, with N being an even integer, giving both programs equal opportunity to play first. The evaluation phase will require a lot of matches so please be mindful of your program's runtime. We will perform a screening process by pairing your agent with a random agent to ensure the run-times matches the expectations (check the Tournament Constraints in Section 5.1.1).

5.1.1 Tournament Constraints

During the tournament, we will use the following additional rules:

- **Execution Environment**. We will run the tournament on SOCS teaching Linux environment (as in the mimi.cs.mcgill.ca server and desktops in Trottier 3rd floor). These run Linux, Python Python 3.10. We will only install libraries as defined in `requirements.txt`, so you are not allowed to use external libraries. This means built-in libraries for computation are allowed but libraries made specifically for machine learning or AI are not. If you think you would require some external libraries not specified in `requirements.txt` and which is not a AI/ML specific library, please post a question in Ed to get an approval from the TAs, but we will almost always disallow things unless you really convince us we missed something needed.
- **Turn Timeouts**. During each game, your agent will be given no more than 2 seconds to choose its move. Please note that in past years, extra time was given for the first move, but it was found that no strong players used this time, and it makes marking much longer, so now the rule is 2 seconds each and every move. Use the code provided to see your agent's current timings, and consider using the time library already imported into student_agent for you to terminate your search on time. If your agent exceeds the time limit drastically, you will lose some fraction (up to **all of them** for infinite loops) of your performance score.
- **Illegal moves**. In the game, if your agent attempts to move to positions which are illegal, i.e. a square that isn't empty or a move that is too far away, then the game code makes a random move for you. If your code fails during execution, then the game will also continue with a random move.
- **Multi-threading**. Your agent must be single threaded, run on only one processor and complete all computation when returning from the step function (as in, you are not allowed to "think" on your opponent's time).
- **File IO**. Your player will not be allowed to read and write files: all file IO is prohibited. In particular, you are not allowed to write files, so your agent will not be able to do any learning from game to game.
- **Memory Usage**. Your agent will run in its own process and will not be allowed to exceed 500 mb of RAM. Exceeding the RAM limits will result in a game loss.

You are free to implement any method of choosing moves as long as your program runs within these constraints and is well documented in both the write-up and the code. Documentation is an important part of software development, so we expect well-commented code. All implementation must be your own.

5.2 Report Grading Scheme

The marks for the write-up will be awarded as follows:

- Executive Summary: 5/50
- Detailed explanation: 15/50
- Quantitative Analysis: 15/50
- Pros/cons of Chosen Approach (combined with description of other methods tried, if present): 5/50
- Future Improvements: 5/50
- Organization: 5/50

6. Agent Tips and Heuristics

[Note: Previous tournaments were run on disc flipping games with different movement and flipping rules (e.g. Othello/Reversi), so this advice may not map perfectly to Ataxx. However, it is still a useful starting point. What follows is the advice from last year's tournament.]

Of the algorithms we saw in class, those at the end of the games section are most often *strong* agents in the tournament: alpha-beta and MCTS usually alternate as choices of the top team. In 2022 the winning method was titled "Alpha-Beta Pruning with Move Order and Don't Lose Evaluation Heuristic and Game-State Memoization".

However, as your time in the end of term is precious, know that some teams placed in the top half of scores without any large search, simply by coding particularly clever heuristics with greedy approaches. All students we've looked at in the top 20% use methods based on efficient tree search.

Ideas for heuristics to get you started are contained in the greedy corners agent. You can also check online to see what strategies human players have used (please cite them if you use them).

7. Academic Integrity

This is a group project. The exchange of ideas regarding the game is encouraged, but sharing of code and reports between different groups or paying someone for a solution is forbidden and will be treated as cheating. We will be using document and code comparison tools to verify that you were truly in charge of creating your own submission.

Please see the syllabus and www.mcgill.ca/integrity for more information.

8. Contact

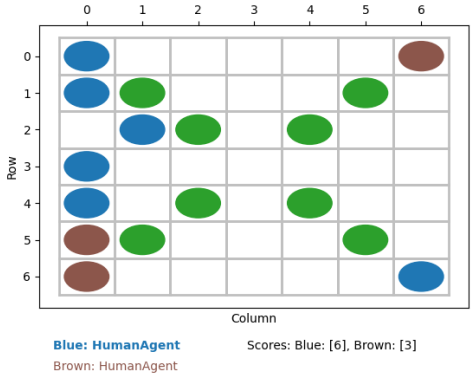
Please post on Ed with the Projects label with any questions.

9. Acknowledgements

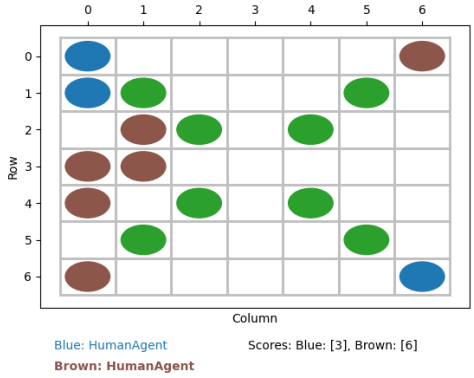
This document is adapted from David Meger's previous final project specification: `https://github.com/dmeger/COMP424-Fall2024`
Thanks to Dr. Meger for sharing his code and specification! Last year's specification document available at: `https://www.overleaf.com/read/vnygbjryrxrt#7b70cb`

(a) Step 0

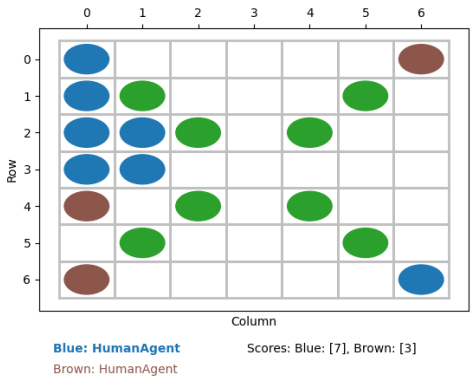
(b) Step 1



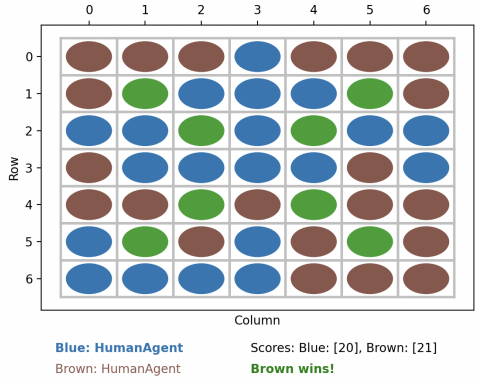
(c) Step 2



(d) Step 3



(e) Step 4



(f) Step 5

Figure 5: A sample game between two human agents on the Big X board layout. The board is initialized (Step 0) with symmetrical player piece placements (brown and blue) and barriers (in green). Blue and Brown both duplicate for two rounds (Step 1). Then, Blue duplicates from $(2, 1)$ to $(3, 0)$ (Step 2), then the Brown jumps two tiles from on $(5, 0)$ to $(3, 1)$ (Step 3). Blue responds by duplicating from $(1, 0)$ to $(2, 0)$ (Step 4). Fast forward and Brown wins the game with an example endgame shown (Step 5).